Lecture 14: Bagging & Boosting

Bootstrapping

- Given a set of data that are independent and identically distributed (i.i.d.) samples, D = {Z_1, Z_2,…Z_n}, we want to compute θ

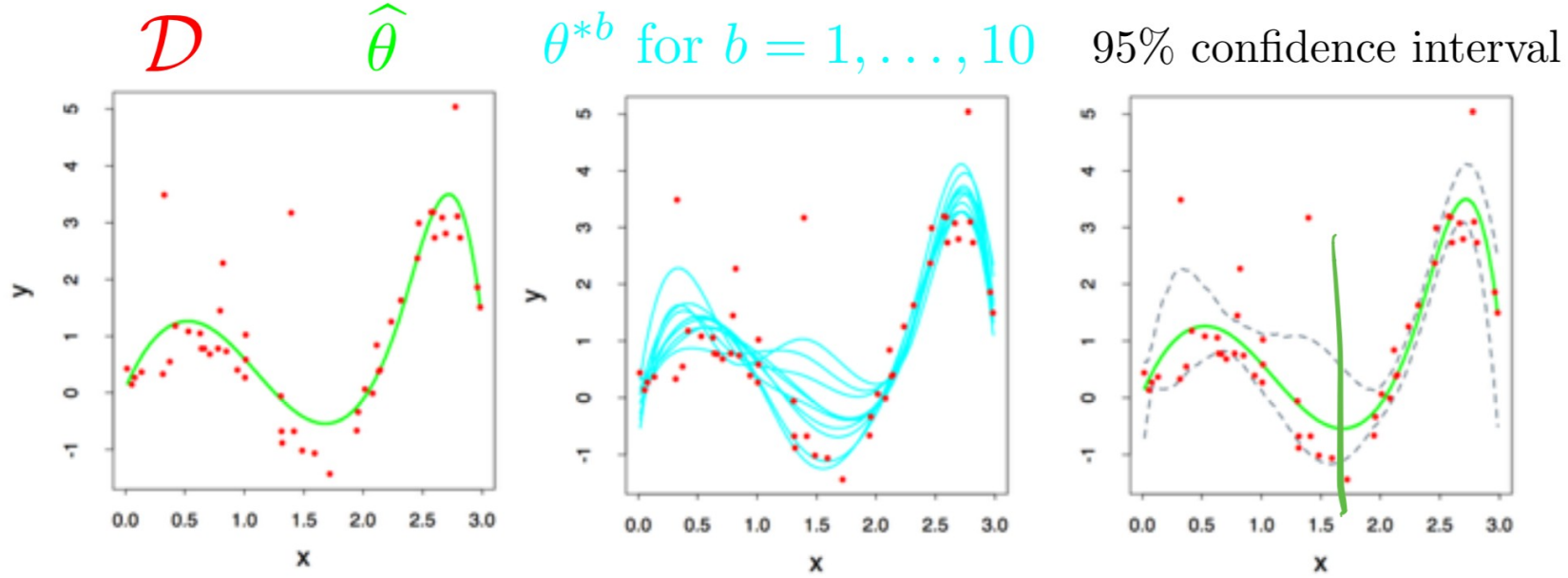
- For b=1, 2,…B, we define the *b*th bootstrapped dataset as drawing n samples ***with replacement*** from D

$$\mathcal{D} = \{z_1, \ldots, z_n\} \overset{i.i.d.}{\sim} F_Z \qquad \widehat{\theta} = t(\mathcal{D})$$

$$\mathcal{D}^{*b} = \{z_1^{*b}, \ldots, z_n^{*b}\} \overset{i.i.d.}{\sim} \widehat{F}_{Z,n} \quad \theta^{*b} = t(\mathcal{D}^{*b})$$

Bootstrap applications

- Common applications of the bootstrap:
  - Estimate parameters that escape simple analysis like the variance or median of an estimate
  - Confidence intervals
  - Estimates of error for a particular example:



Bootstrapping vs k-fold cross validation

- Bootstrapping is effectively randomly sampling *with replacement*, while in k-fold cross validation everyone gets a turn as part of a validation dataset

Bootstrap takeaways

- Advantages:
  - Bootstrap is very generally applicable. Build a confidence interval around anything
  - Very simple to use
  - Appears to give meaningful results even when the amount of data is very small
  - Very strong asymptotic theory (as num examples goes to infinity)

Bootstrap takeaways

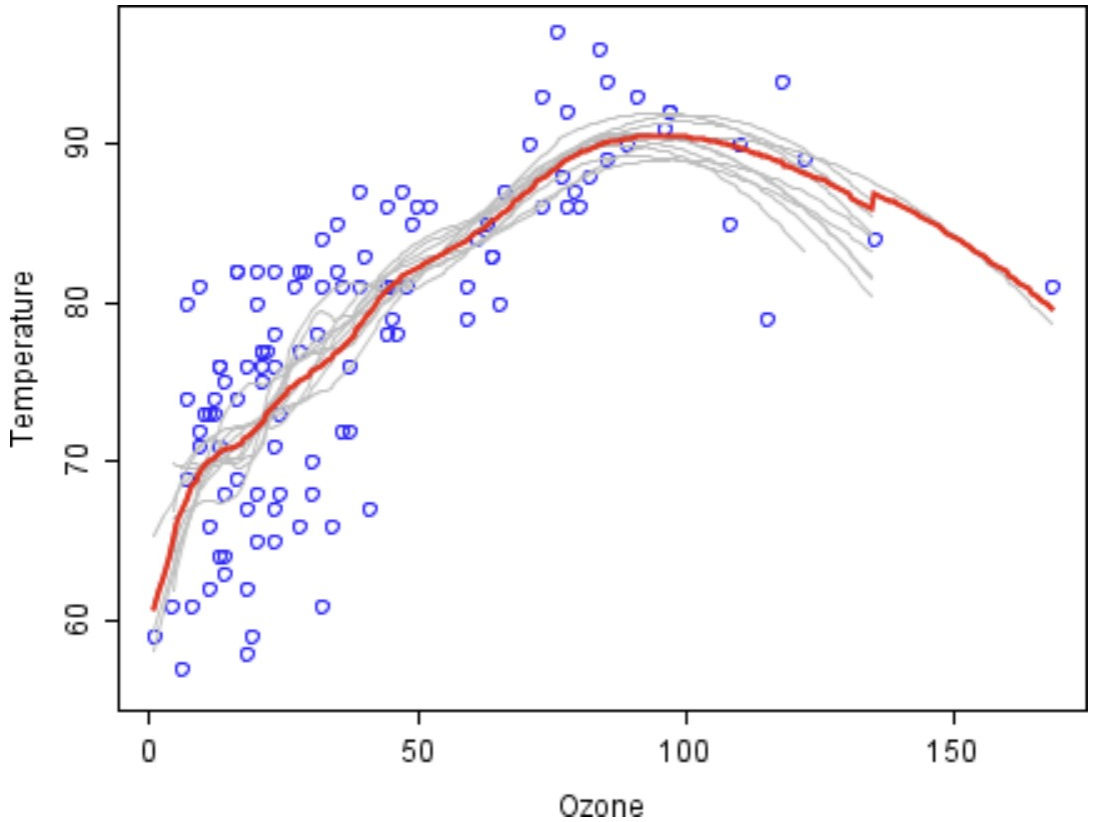
- Disadvantages:
  - Very few meaningful finite-sample guarantees
  - Potentially computationally intensive
  - Reliability relies on test statistic and rate of convergence of empirical CDF to true CDF, which is unknown
  - Poor performance on “extreme statistics” (e.g., the max)

Bagging

- Stands for “**b**ootstrap **agg**regat**ing**”
- Works with both classification and regression algorithms
- Reduces variances
- Helps avoid overfitting
- Very often applied to decision tree methods, but generalizable to any method
- Is a “model averaging” approach

Bagging

- Basically, the goal of bagging is to average over many bootstrapped learners to get a more representative fit



Bagging

- You’ve essentially already learned bagging!
  - Random forests are averages over several decision trees, so essentially an “aggregate” tree

- Any time that you can average your learned variables (choice of average is up to you!), you can use bagging

Boosting

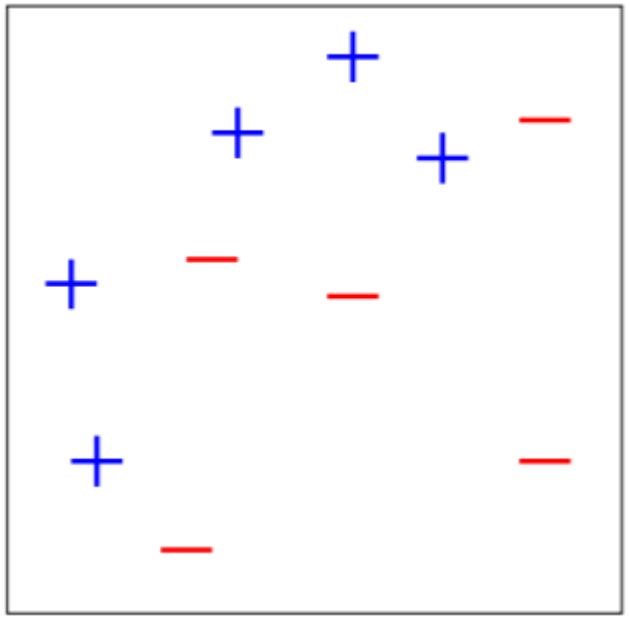
- Motivated by the question “can a set of weak learners create a single strong learner?”

- Similar to, but different from, bagging:
  - Bagging *averages* many low-bias, lightly dependent classifiers to **reduce variance** (bootstrapping is a low-bias technique for validation)
  - Boosting *learns* linear combinations of high-bias, highly dependent classifiers to **reduce error**

- We will first look at boosting visually

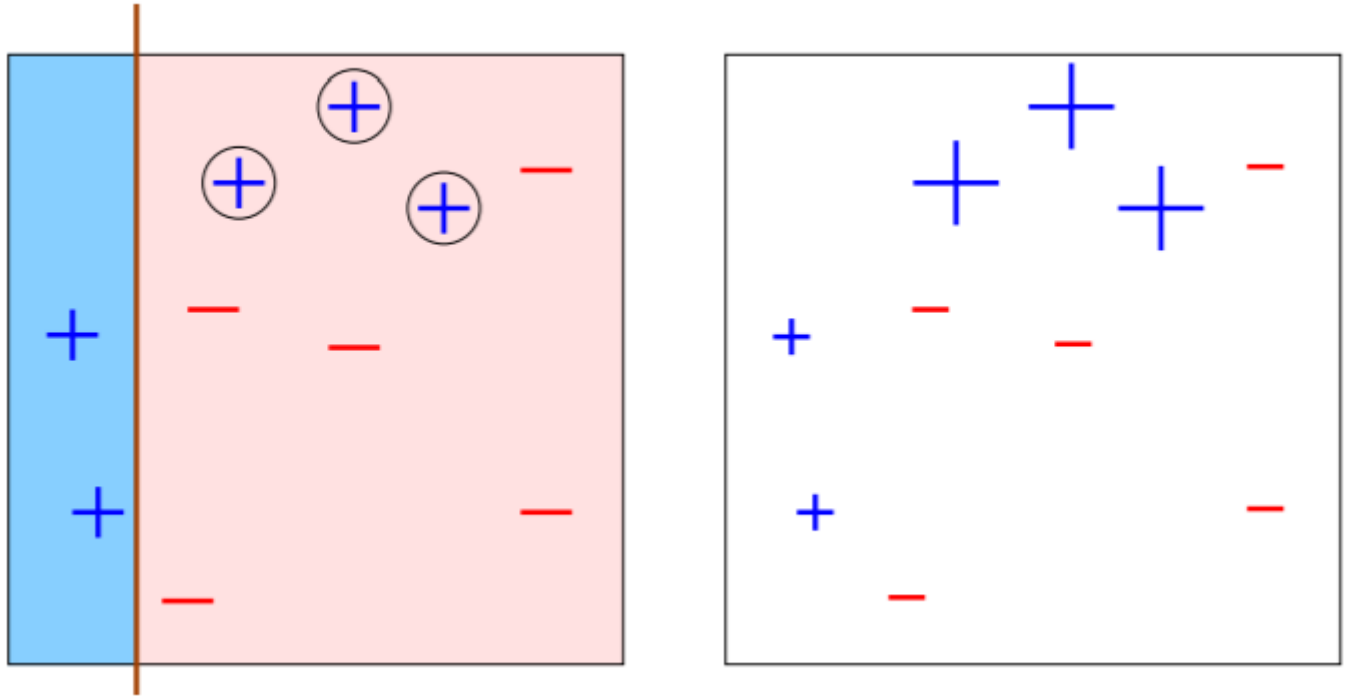
Boosting

- Start with 2 features, 10 data points:
  - Data clearly not linearly separable
  - All data points have equal weight



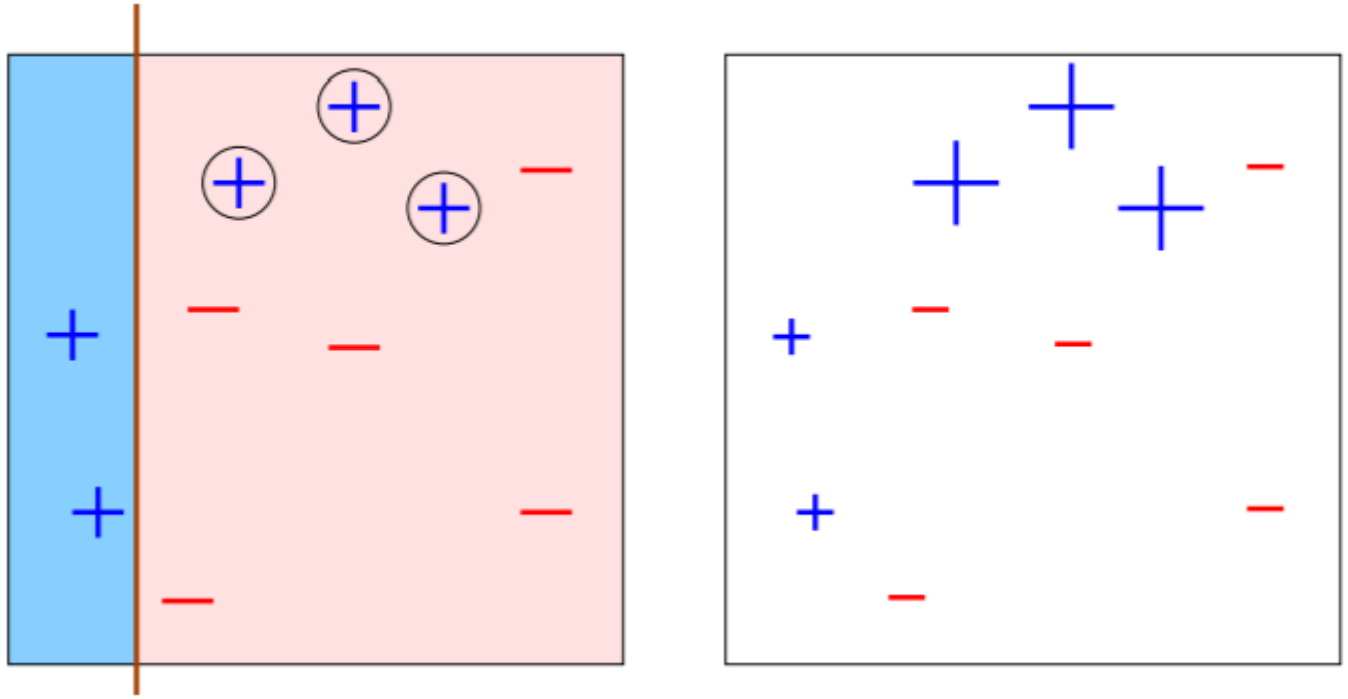
Boosting – step 1

- Make a “base classifier” – do the best you can to separate the data with horizontal or vertical lines (“decision stumps”)



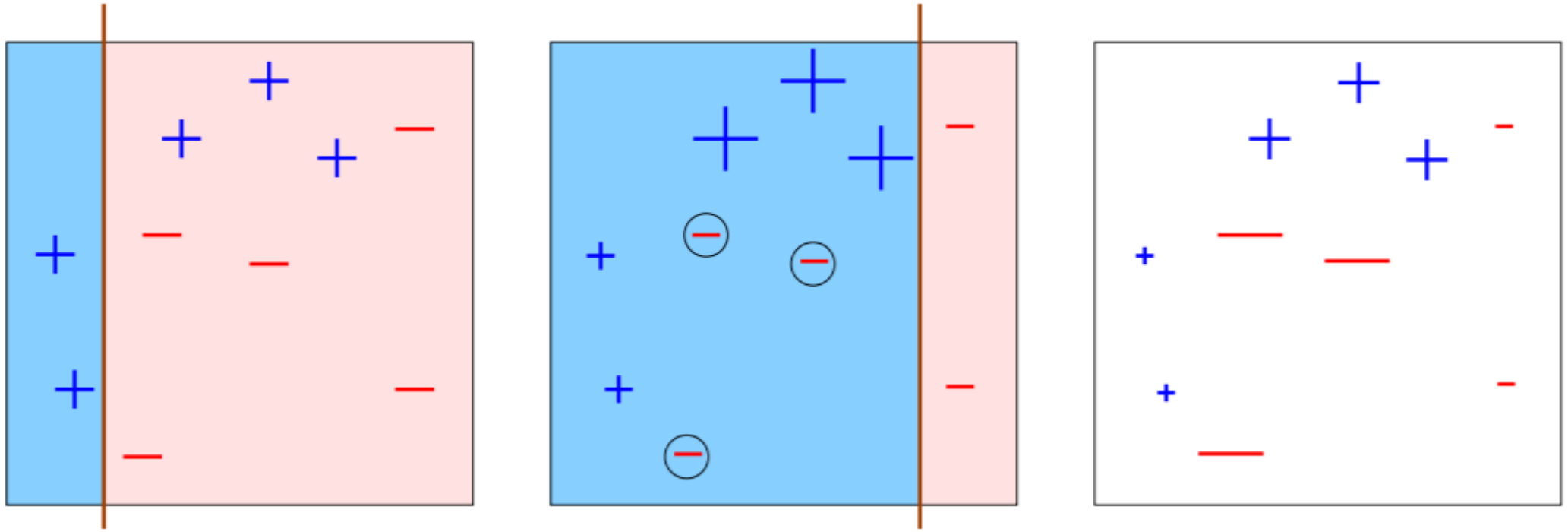
Boosting – step 1

- 3 misclassified points (error 30%) – those receive more/larger/bigger weight (shown here as being larger)



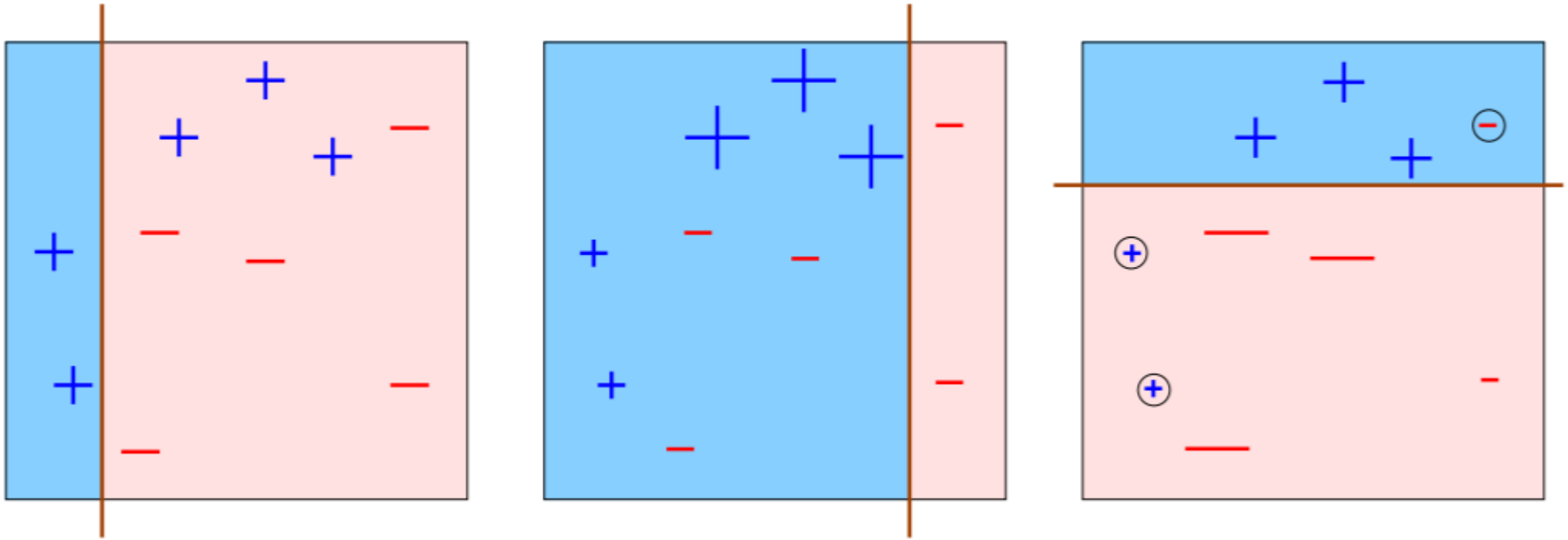
Boosting – step/round 2

- 3 misclassified points (*error is 0.21*) – those receive more/larger/bigger weight (shown here as being larger)



Boosting – step/round 3

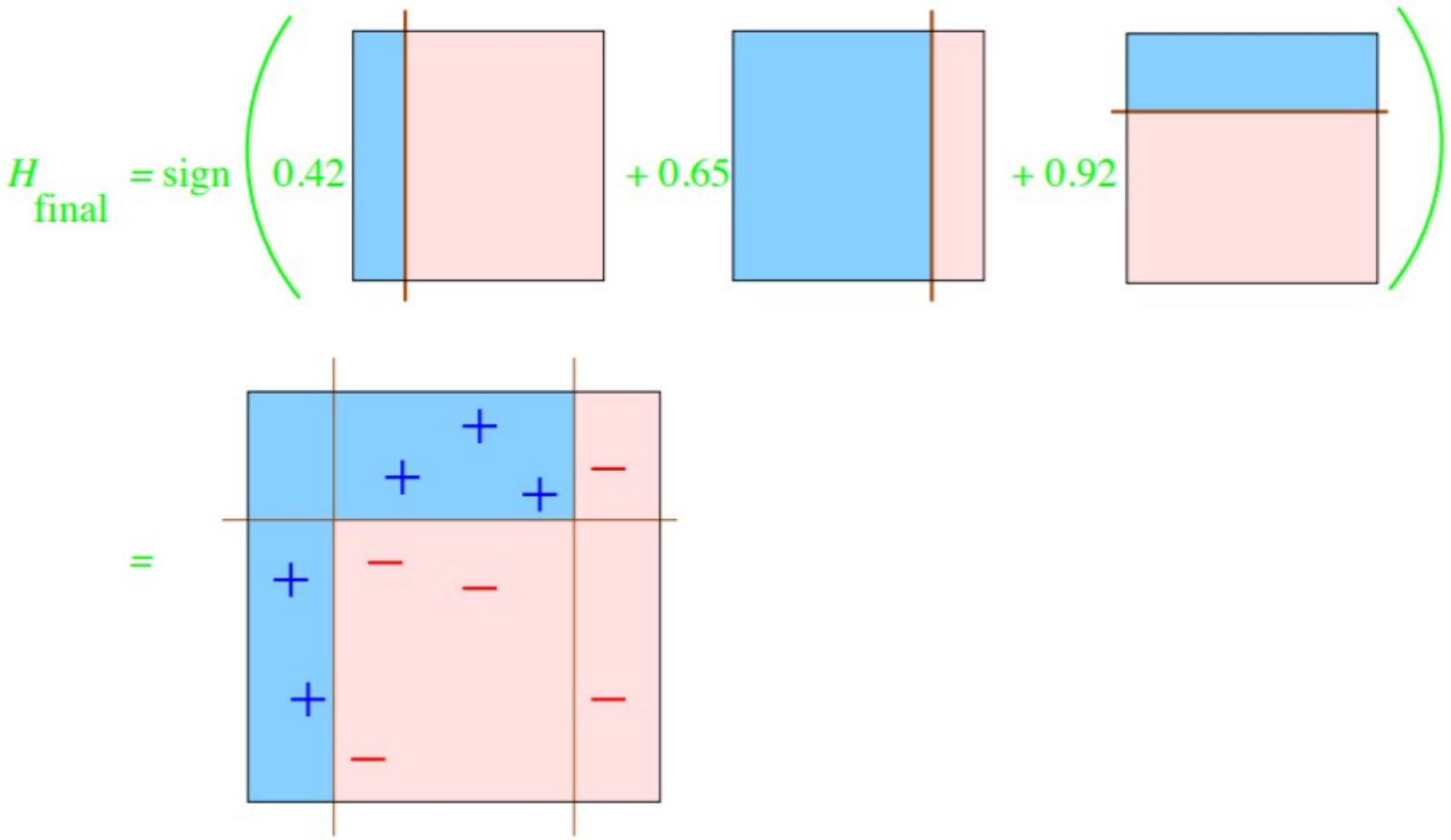
- 3 misclassified points (*error is 0.14*) – those receive more/larger/bigger weight (shown here as being larger)



Boosting

- Since you’ve successfully classified these points before, this round’s mistake will hopefully not majorly impact the overall prediction

Final classifier



How to decide where to draw lines?

- Can we simply enumerate all possible lines? Is that computationally feasible?
  - Yes! Can sort data in O(nlogn) time for each feature
  - Evaluate thresholds in O(n) time
  - Overall, quite fast! Makes it attractive in practice as well as in theory